

demand-api-mcp: benchmark, Bedrock, skills and tool descriptions

Oct 1, 2026 · Rafał Galczyński

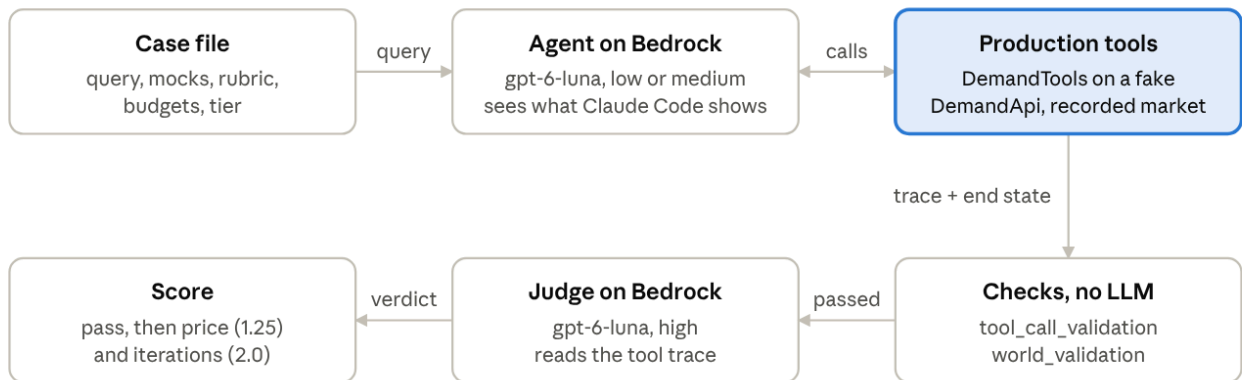
Context

demand-api-mcp is an MCP server that wraps the Cint Demand API in about 25 high-level workflow tools (`search_questions`, `check_feasibility`, `create_target_group_draft`, `transition_target_groups`, `diagnose_deliverability`, `update_target_group`, ...) plus one meta-tool, `load_skill`. Every MCP client receives every tool's name, description and input schema on every turn, so the size and wording of that text is a cost and a quality lever at the same time. The benchmark exists to measure both: it lets a real LLM drive the shipped tools against a fake Demand account and scores each answer for correctness, price and number of rounds.

Bedrock and benchmark integration

Since PR #237 (DEX-5134), the benchmark runs agent and judge on Amazon Bedrock, using OpenAI `gpt-6-luna` through the Converse API. Gemini is still configured as a fallback. The harness, provider clients and scoring come from `java-mcp-commons` (`benchmark/` module, Bedrock support added in DEX-4989). This repo only supplies the cases, the fake world and the run configurations.

Every attempt drives the shipped tools against a fake world, judged on Bedrock



The only part that is not production code is the API underneath the tools. Failing a check ends the attempt before the judge is called.

How one attempt works

1. The harness builds a fresh fake world from the case's `mocks` and the market recorded through LFS. It refuses to start unless every tool can run detached (`Detachment preflight OK`).
2. The agent model gets what Claude Code would show it (`client.profile: claude_code`): the server instructions, plus each tool's name, description and input schema. It never sees `outputSchema`.

3. The tools are the **production** DemandTools handlers running on a state-backed fake DemandApi. That is the only class that leaves the process, so a score measures shipped behaviour.
4. tool_call_validation minimums and world_validation (end-state checks, no LLM involved) run first. Then the judge (gpt-6-luna, reasoning high) reads the tool trace and checks it against the case's text_validation rubric.
5. A passing attempt is scored on price and iterations against expected_min, with weights price 1.25, iterations 2.0 and non_reasoning_time 0.

Model tiers. Every case runs on tier base (luna, reasoning low). A case moves to tier: extra-think (reasoning medium) only when base scores it under 0.8 on tool text that already steers it correctly.

```
# benchmark/demand/global.yaml (excerpt)
models:
  tiers:
    - { id: base,          provider: bedrock, model: "global.openai.gpt-6-luna", thinking_level:
"low" }
    - { id: extra-think, provider: bedrock, model: "global.openai.gpt-6-luna", thinking_level:
"medium" }
  judge: { provider: bedrock, model: "global.openai.gpt-6-luna", thinking_level: "high" }
scoring:
  pricing: { usd_per_1m_input: 0.30, usd_per_1m_cached_input: 0.03, usd_per_1m_output: 2.50 } #
fixed card
  weights: { price: 1.25, iterations: 2.0, non_reasoning_time: 0.0 }
```

Price is scored at one fixed rate card (gemini-3.5-flash-lite's rates), whatever model served the call. That keeps expected_min.price floors comparable across models. Real spend is still billed at each model's own rate. A case's usd_budget is 3× its floor; the floor is the 85th-percentile scored spend from a 20-attempt run.

Three run configurations

File	Purpose	Attempts per case	Run budget
global.yaml	local full scoring, 34 cases	12	\$10, 10 min
ci.yaml	PR gate, --strict (3 passes needed)	6	\$5, 50 min
holdout.yaml	3 control cases, never read while tuning	10	\$2.70, 13 min

The CI gate. Jenkins scores a PR once it is marked ready for review, with make docker-benchmark-ci, and posts a benchmark commit status that main requires. Drafts are not scored. An attempt lost to the provider (unreachable, content filter, budget) lowers the bar by one; a real failure does not.

Running it

```
git lfs install && git lfs pull          # once: the recorded market lives in LFS
AWS_PROFILE=tech-dev make benchmark-run  # global.yaml
make benchmark-run BENCHMARK_CONFIG=benchmark/demand/holdout.yaml
```

A cheap way to settle one flaky case: a scratch config with only that case, at 12–24 attempts, costs about \$0.02.

What a case looks like (trimmed from `allocationBars_a_supplier_on_a_live_tracker.yaml`):

```
query: >
  In account 5840, our live "Brand Tracker US" target group should stop buying from
  Affiliate Reach. Bar it and keep the rest of the supply as it is.
mocks: { accounts: [ ... projects, target_groups, allocation templates ... ] }
text_validation: >
  A load_skill call with skill "supply-allocation" must appear before the first
  update_target_group call carrying 'allocations' ...
tool_call_validation: { find_suppliers: 1, get_target_group_info: 1, update_target_group: 1 }
usd_budget: 0.15
scoring: { expected_min: { price: 0.05, iterations: 5 } }
```

Gotchas

- Moving to luna made CI slower: about 9–10 minutes against about 3 on Gemini.
- The luna judge has known noise; for example, it misjudges `us_feasibility_quota_split` about 3 times per two runs.
- Never build the benchmark for two worktrees at once. They share the main module in `~/m2`, so the jar can end up with the wrong YAML.

Benchmark roadmap: other models on Bedrock?

Short answer: the plumbing for other models is in place, but no second model is scheduled. As of today every role runs one model, `gpt-6-luna`, with tiers that differ only in reasoning effort. Adding a different model is a configuration change, not code work.

What already supports it

- Each role (every tier, and the judge) has its own `provider`, `model` and `thinking_level`. Moving a role is three lines; the Gemini lines sit commented out under each one.
- Bedrock goes through the Converse API, which is model-agnostic. Any model on Bedrock that supports Converse tool use can be added as a tier, for example `{ id: claude, provider: bedrock, model: "<bedrock model id>" }`, with a matching row under `models.pricing`. A missing pricing row fails at load time.
- Cases choose a tier by name (`tier: extra-think`), so a stronger or different model can be applied to selected cases only.
- Price is scored at one fixed card, so a cheaper or dearer model does not move the price score by its rates, only by the tokens it spends.

What a model change costs

- **Every price floor moves.** The README rule: after an agent-model change, re-read all `expected_min.price` floors from a new 20-attempt run. Never adjust only the case that failed.
- **Tool text tuned for one model may not fit another.** #252 tuned the text and cases for luna at `low` reasoning, and #285 fixed round-ordering behaviour specific to luna. A new model needs its own tuning pass and a holdout check before and after.
- **Judge noise changes with the judge.** Swapping the judge changes what passes even when the agent stays the same.

Library work landed but not adopted yet. `java-mcp-commons` 12.0.0 (DEX-5198) makes the `claude_code` profile cut descriptions at 2,048 characters, the way Claude Code does. DEX-5182 adds a `context_footprint` score dimension. DEX-5203 adds gates on minimum mean score and maximum cost per case. `demand-api-mcp` is still on 11.8.0. Adopting 12.x will change scores, because 12 of our 26 tool descriptions are longer than the cut (see the descriptions section).

Open questions for the team (no ticket found here; Jira was not reachable from this session):

- ☐ Do we want a matrix run, for example luna plus a Claude model on Bedrock, as a robustness check outside the PR gate?
- ☐ Which model is the PR gate's model of record? Floors can be calibrated for only one.
- ☐ Is the Bedrock rate card for the `global`. luna profile confirmed? `global.yaml` notes that it is not.

Skill loading (`load_skill`)

A skill is long guidance, usually how to read a tool's reply field by field, that is served as a **tool result** on request rather than sent in the catalogue every turn. Only conversations that need it pay for it. It shipped in #238 (DEX-4468), moved into `java-mcp-commons` as standard Agent Skills in DEX-5116 (#279), and is used by three skills today.

Skill	Governs	Size
diagnose-deliverability	<code>diagnose_deliverability</code>	47.0k chars
supply-allocation	<code>get_account_allocation_template_association</code> , <code>get_allocation_template</code> , <code>create_allocation_template</code>	16.9k chars
allocation-templates	<code>list_allocation_templates</code> , <code>get_allocation_template</code> , <code>create_allocation_template</code> , <code>find_suppliers</code>	6.0k chars

Where the parts live

- **The skill:** `src/main/resources/skills/<name>/SKILL.md`, a standard Agent Skills file. The tools a skill covers are listed under `metadata.governs`. Registered in `DemandTools.SKILLS`.
- **The pointer:** the library adds a sentence at the *start* of each governed tool's description, so it survives Claude Code's 2,048-character cut.
- **The index:** the skills section opens the server instructions, and it also ends `load_skill`'s own short description. That way it still reaches the model through a gateway that drops the instructions.
- **The loader:** `load_skill(skill: enum)`. The input is a closed list of skill names, and the reply is the skill body only. It has no output schema.

What the model sees. The skill file:

```

---
name: supply-allocation
description: >-
  What supply a target group buys through and how a saved allocation's entries are read ...
  Load it before answering what supply a target group is on ... and before sending allocations.
metadata:
  governs: get_account_allocation_template_association, get_allocation_template,
  create_allocation_template
---
What a target group buys through, and how a saved allocation is read. ...
## supply_allocation fields
- `supply_allocation.bounds_nothing` (boolean, nullable): True when nothing narrows the supply
...

```

The governed tool's description, as published:

```

Before calling create_allocation_template, call load_skill with skill "supply-allocation" and
"allocation-templates".

Save a new supply allocation on an account: the reusable supply create_target_group_draft takes
as ...

```

The call and its reply on the wire:

```

// tools/call
{ "name": "load_skill", "arguments": { "skill": "diagnose-deliverability" } }
// result: the SKILL.md body as text, which stays in context for the rest of the conversation
{ "content": [ { "type": "text", "text": "Diagnose why a LAUNCHED target group is not filling:
..." } ] }

```

Expected flow for "why is my tracker getting no completes?":

1. Round 1, in parallel: `get_connection_info`, `list_target_groups`, and `load_skill("diagnose-deliverability")`.
2. Round 2: `diagnose_deliverability`. It returns `needs_input: true` with questions only the survey owner can answer.
3. Ask the user the questions and stop. Then call again with `client_confirmations` set to their answers.

Rules we settled on

- A rule the model needs *before* the call, or *regardless of* the reply, stays in the description: when to call, what never to claim, the two-phase `needs_input` gate. Field-level reading moves to the skill.
- A load needs no other tool's reply, so it goes in the **same round** as the first reads, never in a round of its own (#285, DEX-5246). Without this, luna sent each load alone and `allocationBars` took 6–9 rounds against a clean path of 5. Its score went 0.375 → 0.885. The wording is in DemandTools.SKILLS' loader description and in the round rule in `workflow-mcp.md`.

- A test holds each skill to every field of the reply. A benchmark case can require the load: `allocation_bars` fails if `update_target_group` with `allocations` comes before `load_skill`.
- `/mcp-beta` serves a narrower tool set. `Skills.forTools` narrows the skills to match, so no pointer names a skill the endpoint cannot load.

Measured effect. On the #229 allocation work, CI went from *budget exhausted* (\$5.01, 71 attempts disqualified) to passing at about \$4.0–4.1. The catalogue shrank from 331.6k to 266.8k chars. We also tried moving nine more tools' reply docs into skills and **rejected it**: `load_skill` calls rose from 42 to 244, turns per attempt rose, prompt tokens stayed flat, and the run went over budget. A skill pays off only for a long reading that few conversations need.

Description limits and how we work around them

There are two hard limits. **Claude Code cuts each tool description, and the server instructions, at 2,048 UTF-16 units** and appends ... [truncated]; the model cannot fetch the rest. **Every tool's text is resent on every turn**, so each character is paid for again in every round of every conversation. Input schemas and tool results are not cut, which is why long guidance belongs in a skill.

Where we stand. 12 of 26 tool descriptions exceed the cut, before the skill pointers are even added. So does the instructions file, `workflow-mcp.md`, at 6.3k characters.

Tool	Description chars
<code>create_target_group_draft</code>	5,924
<code>check_feasibility</code>	4,653
<code>search_questions</code>	4,491
<code>transition_target_groups</code>	4,145
<code>get_target_group_info</code>	4,009
<code>update_target_group</code>	3,530
<code>update_pacing</code> , <code>get_connection_info</code> , <code>check_feasibility_batch</code> , <code>diagnose_deliverability</code> , <code>list_allocation_templates</code> , <code>list_profile_templates</code>	2,121–2,780

Today's benchmark (library 11.8.0) still shows the model the full text. Library 12.0.0 makes the `claude_code` profile apply the cut, and `claude_code_full_text` runs without it, so running a suite under both shows what the cut costs us. Expect scores to move once we adopt 12.x.

What we have done about it

1. **Output schema out of the description** (#237, DEX-5199, library DEX-5201). Tools are built with `DescriptionMode.OPERATION_ONLY`, so the response schema is published only as the MCP `outputSchema`. The description is rules alone. The catch: the benchmark's model, like most clients, never sees `outputSchema`, so guidance written on a reply field reaches nobody. Put it in the description or a skill.
2. **Skills for long readings** (previous section). This moved 58.8k chars of `diagnose_deliverability` field docs, and then the allocation docs, out of the per-turn text.
3. **Must-see text first.** Skill pointers open the description, and the skills index opens the instructions. The library keeps the index within 1,800 chars (2,048 less room for a gateway header) and refuses at build time an index that would not fit.

4. **Rules in the description, facts in the reply.** A reply reports what happened and carries no advice. When to call a tool, what comes next and what never to claim live in its description (`doc/tool-output-contract.md`).
5. **Measured trims, not blind ones.** For example, #211 cut the rendered catalogue from 314k to 280k chars with every pinned phrase kept. Tests pin the phrases cases depend on, because removing them has regressed cases. Two examples from the `load_skill` trial: dropping "A launched target group's audience and supply are fixed" and "get_target_group_info says how much, this says why" sent the audience-swap case to 8/15 and `nothing_wrong` to 11/15, so both went back.

Lessons that cost us a run or two

- **Shorter is not always cheaper.** Gemini cached prompts in whole 16,384-token blocks. A trim that drops a turn below a block boundary can *raise* cost, and two trims lowered the cache share. Compare cached and uncached tokens separately before blaming the text. This was measured on Gemini; it has not been re-checked on luna.
- **The compact output-schema render** (a field list instead of pretty JSON) cut prompt tokens by 20% but lowered the cache share, with scores within noise. It became moot once the schema left the description.
- **Don't tune against the holdout.** Score `holdout.yaml` before and after a round of description tuning. If `global.yaml` rises and the holdout stays flat, the text was fitted to the cases rather than making the product better.
- **Small wording moves round counts.** In #285, a narrower, hedged version of the round rule brought the extra rounds back. Only the firm wording ("send it in the same round as the first reads") fixed them.

Where to read more

- `benchmark/README.md`: models, CI gate, price floors, holdout, recordings, case anatomy.
- `benchmark/demand/global.yaml`, `ci.yaml`, `holdout.yaml`: the run configurations.
- `doc/tool-output-contract.md`: what goes in a description and what goes in a reply or a skill.
- `src/main/resources/skills/*/SKILL.md` and `DemandTools.SKILLS`: the skills and the loader wording.
- `src/main/resources/mcp/workflow-mcp.md`: the server instructions, including the round rule.
- `java-mcp-commons README`, sections "Why the defaults place the text where they do" and "The client view": the 2,048 cut and the skill placement.
- PRs: #237 (Bedrock and luna, output schema out of descriptions), #238 (`load_skill`), #246 (supply-allocation skill), #252 (text tuned for luna), #285 (same-round loads), #211 (catalogue trim).
- Make targets: `make benchmark-run`, `make benchmark-ci`, `make docker-benchmark-build`, `make docker-benchmark-run`, `make benchmark-capture` (manual, needs credentials). Run `make help` for the full list.